Khulna University of Engineering & Technology (KUET)
Department of Computer Science and Engineering

CSE 4102 · Computer Graphics and Image Processing Laboratory
4th Year, 1st Term

The Ramen Shop

A Cozy Japanese Ramen Restaurant in OpenGL



The shop at night, rendered in real time by the project.

Submitted by
Rajorshi Das
Roll: 2107060
Department of Computer Science
and Engineering, KUET

Submitted to
Md Tajmilur Rahman
Lecturer
Department of Computer Science
and Engineering, KUET

Md. Mubtashim Abrar Nihal
Lecturer
Department of Computer Science
and Engineering, KUET

Built with C++17 · OpenGL (fixed function + GLSL 1.20) · FreeGLUT · GLEW
October 2026

Contents

1 Introduction . . . **1**
- 1.1 Objectives . . . 1
- 1.2 Tools . . . 1
- 1.3 Proposed features and what was implemented . . . 1
- 1.4 Sources followed . . . 1

2 The Shop and Its Surroundings . . . **1**

3 Methodology . . . **3**
- 3.1 Program structure . . . 3
- 3.2 Modelling from basic shapes . . . 3
- 3.3 The objects of the shop . . . 3
- 3.4 Trees, plants and the landscape . . . 3
- 3.5 3D transformations . . . 5
- 3.6 Textures and texture mapping . . . 5
- 3.7 Illumination: ambient, diffuse and specular light . . . 8
- 3.8 Glossy surfaces and the specular term . . . 8
- 3.9 Sky, day and night . . . 9
- 3.10 Gouraud and Phong shading . . . 11
- 3.11 Shadows . . . 11
- 3.12 Glass refraction (screen space) . . . 12
- 3.13 Ray tracing . . . 12
- 3.14 Animation . . . 13
- 3.15 Cameras, interaction and controls . . . 14
- 3.16 The ramen cooking game . . . 15
- 3.17 Keeping it fast . . . 16

4 Results . . . **16**

5 Discussion . . . **16**

6 Conclusion . . . **17**

List of Figures

1 Floor plans of the two storeys . . . 2
2 The rendering pipeline: work done once at start-up (left) and in every frame (right). . . . 3
3 The construction primitives: every object in the shop is some combination of these. . . . 3
4 Some of the objects of the shop, each built from the primitives of Figure 3. . . . 4
5 The upstairs tatami room. . . . 4
6 The kinds of tree and the undergrowth, in daylight. . . . 5
7 Buildings, water and street lighting of the landscape. . . . 6
8 Parent–child transformations . . . 6
9 The procedural textures as they appear in the scene. . . . 7
10 (a)–(e) The illumination terms switched off one at a time, at night: without ambient the shadows are slightly deeper, without diffuse only glow and highlights remain, without specular the shiny counter goes flat. (f) The same view with Gouraud shading. . . . 8
11 Every light switched on on its own at night, in the same view (the moon is off). Only the surfaces near a lamp are lit by it. . . . 9
12 The specular lobe $(\cos\theta)^n$ for four materials, where θ is the angle between $\mathbf{N}$ and $\mathbf{H}$: a larger n makes a narrower highlight. . . . 9
13 The lacquered counter top with and without the specular term, looking down at the lamps. The highlight is the mirror image of the lamp in the lacquer. 10
14 The sky by day and at night. . . . 10
15 The same views drawn with per-pixel Phong shading and with per-vertex Gouraud shading (key G). . . . 11
16 Top: how a tree is projected onto the ground, and the steps of soft shadow mapping. Bottom: projected tree, bicycle and person shadows; the dining area without and with the shadow map. . . . 12
17 The tumbler on the left table without and with screen-space refraction. . . . 12
18 Ray tracing steps . . . 13
19 Ray tracing (key R): the steel pots mirror the shop, the glass ball refracts the counter, and the mirror reflects the room. Up close one ray covers several pixels, so the result looks slightly blocky. . . . 13
20 The moving things in the street and the glass wind chime under the awning. . . . 14
21 Six of the seven camera modes. . . . 15
22 The game: its state machine and three screens (HUD drawn with an orthographic projection). . . . 15
23 The finished scene. . . . 16

List of Tables

1 Tools and size of the project. . . . 1
2 Proposed features and their final status. . . . 1
3 The source followed for each technique. The code of the scene, shaders, textures, ray tracer and animation is written for this project. . . . 2
4 What is in each zone of the scene. . . . 2
5 Modules of the program. . . . 3
6 The main objects of the shop and how each is built. . . . 4

7 The tree types, how many of each are placed, and how they differ. 5
8 Where each 3D transformation is used. 6
9 The procedural textures. The cobblestone street is built from geometry with a hashed stone colour. 7
10 The eight lights. At night all fixtures glow in their own warm colour and flicker slowly. 8
11 The light values of each fixture. 8
12 Specular strength and shininess of the materials of the shop. 9
13 The layers of the sky. 10
14 Gouraud and Phong shading compared as implemented here. 11
15 The ray-traced objects. All other shapes (floors, walls, counter, tables, lamps) exist only so that rays have something to hit. 12
16 The moving objects of the scene. 13
17 The seven camera modes. 14
18 The keyboard and mouse controls. 14
19 The four stations of the game. 15
20 The optimisations used to keep the scene interactive. 16
21 Scene statistics. 16
22 Parameters that can be changed to see their effect. 17

1 Introduction

A small ramen-ya sits on a quiet Japanese street. Its two-storey wooden shop has a dining counter with an open kitchen, two small tables, a cosy tatami room upstairs, a street with sidewalks, cherry and maple trees, and a lake with ducks behind it. This project builds that whole place as an interactive, real-time **OpenGL** program. You can fly around it with seven cameras, switch every lamp on and off, change between day and night, click the doors open, and play a small ramen-cooking game.

Everything is made in code. Every object is built from a few basic shapes, and every texture is painted by the program itself at start-up, so there are no model or image files to load. On top of the lighting and shading there are shadows, a refractive glass, a small ray tracer, and many moving things: a cyclist, a person, a cat, a rippling banner and a glass wind chime.

1.1 Objectives

1. Model a believable scene (shop, upstairs room, street, nature) from basic shapes and 3D transformations.
2. Light it with ambient, diffuse and specular light from several lights that can be switched on and off.
3. Use procedural textures, and shade with both Gouraud and Phong shading.
4. Add shadows, glass refraction and real ray tracing.
5. Make the scene alive: animation, several cameras, mouse and keyboard interaction, and a small game.

1.2 Tools

Language and build	C++17, Visual Studio (MSBuild), x64 Debug and Release
Graphics API	OpenGL fixed-function pipeline plus GLSL 1.20 shaders (Phong, glass refraction, ray-trace compositing)
Libraries	FreeGLUT (window, input), GLEW (extensions), GLU (quadrics, matrices)
Size	about 15,000 lines in 18 source files; no model, image or sound files

Table 1: Tools and size of the project.

1.3 Proposed features and what was implemented

Proposed feature	Status	In the final project	Sec.
Translation, rotation, scaling	Implemented	cyclist, person, cat, doors, fans, trees	3.5
Hierarchical modelling	Implemented	cyclist, person, cat, wind chime, trees	3.5
Perspective camera	Implemented	45° field of view, seven cameras	3.15
Orthographic view	Adapted	used for the game screens (HUD)	3.15
Ambient, diffuse, specular light	Implemented	8 lights, each term can be switched off	3.7
Flat shading	Not implemented	only smooth shading is used	3.10
Gouraud shading	Implemented	key `G`	3.10
Phong shading	Implemented	own GLSL shader	3.10
Skybox	Implemented	dome, sun, moon, stars, clouds	3.9
Textures	Implemented	13 procedural textures	3.6
Added beyond the proposal			
Shadows	Implemented	stencil shadows and soft shadow map	3.11
Glass refraction	Implemented	the table tumbler, key `Z`	3.12
Ray tracing	Implemented	pots, bowls, glass ball, mirror, key `R`	3.13
Animated characters and physics	Implemented	cyclist, person, cat, wind chime	3.14
Ramen cooking game	Implemented	state machine and HUD, key `Enter`	3.16

Table 2: Proposed features and their final status.

1.4 Sources followed

2 The Shop and Its Surroundings

One world coordinate system is used: x runs to the right, y points up and z points toward the street. One unit is about one metre. The ground floor is 9.8×7.8 units and 3.3 high; the upper floor adds 2.0 and the gable roof another 2.2. Behind the shop is a large lawn (240 units wide) with a lake at $(0, -24)$; in front are the road and two sidewalks at $z \approx 7$ to 17.

Technique	Source or reference followed	Ref.
Ambient, diffuse, specular light	Phong reflection model, Lambert's cosine law	[1]
Specular with half vector	Blinn–Phong	[2]
Gouraud shading	colour interpolation over a polygon	[3]
Ray tracing	Whitted's recursive ray tracing	[4]
Shadow map and soft edge	Williams; Reeves, Salesin and Cook	[5], [6]
Fresnel reflection of glass	Schlick's approximation	[7]
Clipping of wall pictures	Sutherland–Hodgman algorithm	[8]
Matrices, projection, stencil, blending	OpenGL Programming Guide, GLSL 1.20 specification	[9]
Procedural textures	value noise and fractal Brownian motion	[10]

Table 3: The source followed for each technique. The code of the scene, shaders, textures, ray tracer and animation is written for this project.



Figure 1: Floor plans (the street side of the shop is at the bottom). The tatami room is covered by 8 rows of mats in a staggered pattern.

Zone	Contents
Ground floor	dining counter with 6 stools and a lacquered top; four ramen bowls with steam; open kitchen (two steel stock pots, hood, shelves, fridge, sink); two tables with two chairs each and a lamp over each; paper box lanterns, hanging lanterns, floor lantern tower, pendant glass lamps, area-light panel; noren curtain, wall clock, shoji doors and windows
Upper floor	tatami floor with cloth borders, plaster walls with timber frame, fusuma doors, board ceiling; futon bed, chabudai tea table with tea set, cushions, tansu chest, sideboard with radio, mirror, six wall pictures, windows, ceiling fan and stand fan
Outdoors	cobblestone road, two flagstone sidewalks, 12 street lamps, 10 houses, cherry, maple, pine and broadleaf trees, bamboo, flowers, stone lanterns, lake with bridge, torii gate and two ducks

Table 4: What is in each zone of the scene.

3 Methodology

3.1 Program structure

The program is split into small modules (Table 5). At start-up it builds everything it needs once; every frame it then reads input, updates the animation and draws the world (Figure 2).

Files	Role
`main`, `camera`	window, frame loop, timers, the seven cameras, keyboard and mouse input
`shapes`, `texture`	primitives, materials (gloss, emission), procedural textures
`scene`, `exterior`	ground, sky, shop, upstairs room, trees, props, wind chime, shadows
`furniture`, `food`, `decorations`	counter, kitchen, tables, bowls, lanterns, noren, clock
`lighting`, `shader`	the 8 lights, day and night, Phong shader, fog
`shadowmap`, `refraction`	soft shadow mapping, screen-space glass refraction
`rtcore`, `rtscene`, `rtpass`	ray tracer, the shop as the rays see it, threaded pass and compositing
`objects`, `game`	selectable and movable objects, the ramen cooking game

Table 5: Modules of the program.

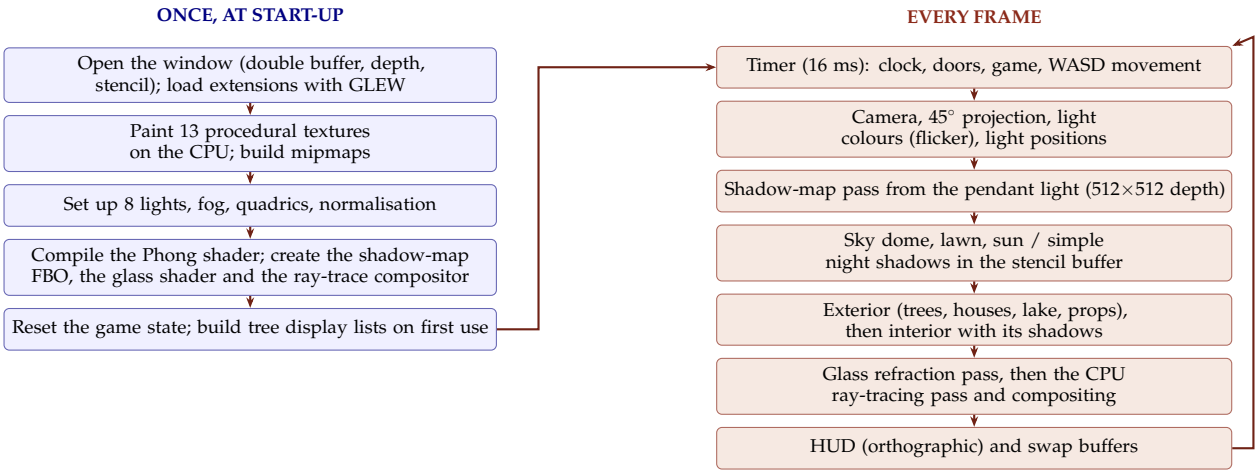


Figure 2: The rendering pipeline: work done once at start-up (left) and in every frame (right).

3.2 Modelling from basic shapes

Every object is assembled from a handful of primitives (Figure 3): a cube, a cylinder and cone (one routine with a bottom and a top radius), a sphere, a torus, planes and wedges. Boxes of other sizes are cubes with non-uniform scale and ellipsoids are scaled spheres. Three more constructions make the special shapes:

- **Lathe.** A profile (r, y) is turned about the y axis: $(r\cos\theta,\ y,\ r\sin\theta)$. Lamp shades, bowls, bottles and the wind-chime bell are made this way.
- **Grid surface.** The nobori banner is a 12×28 grid of quads moved by a travelling wave, calm at the pole and strong at the free edge.
- **Recursive trees.** Each branch grows shorter, thinner child branches; leaves or flowers sit on the tips.

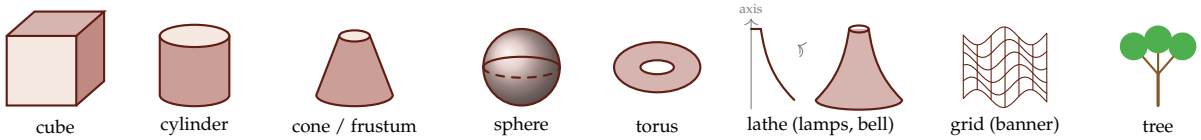


Figure 3: The construction primitives: every object in the shop is some combination of these.

3.3 The objects of the shop

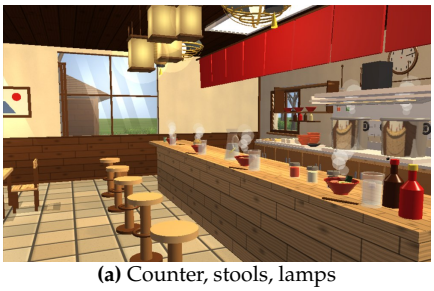
Every object has its own function that builds it from primitives around its own origin, so one call can place, rotate and scale it anywhere. Table 6 lists the main objects and how each is built; Figures 4 and 5 show them.

3.4 Trees, plants and the landscape

The outdoor scene has six kinds of tree (Table 7), shrubs, grass, flowers, a lake with a bridge and ducks, a torii gate, ten neighbouring houses, stone lanterns and twelve street lamps. A branch is two tapered tubes; it grows

Object	Built from	Material and details
Counter	textured box $6 \times 1 \times 0.6$ (body) and a $6.2 \times 0.06 \times 0.8$ top	planks, satin ($n = 40$); top lacquer ($n = 60$)
Stools (6)	base disc, tapered post, torus ring, seat disc	varnished wood; the first one can be moved
Tables, chairs	table: plank top and 4 legs; chair: seat, back, 4 legs, 2 back posts	lacquered tops, dark glossy legs
Kitchen	7 cabinets, 2-burner stove, hood, shelves, fridge, sink with tap, dish rack, bowls, ladle, tongs, wok	steel, tile and wood, each with its own gloss
Stock pots (2)	frustum body, rolled rim torus, two handle tori, mounting plates with rivets, broth disc	dark steel, white specular ($n = 90$), room reflection
Ramen bowl	glazed bowl, broth disc, noodles (6 tori and 3 cylinders), chashu, egg (2 ellipsoids), nori board, 6 onion rings	glaze $n = 110$, broth $n = 85$, egg $n = 42$, nori matte
Lamps	pendant glass lamps (cage and bulb); 5 box lanterns; 2 hanging lanterns (lathe); lantern tower; area-light panel; 2 table lamps (bell lathe, wooden cap)	emission and glow gradients that fade with their lamp
Entrance	2 chochin lanterns (sphere, 13 ribs, gold caps, 14 tassels); noren (9 red cloth boards on a rod); shoji doors and windows (kumiko lattice)	emissive paper; the doors are clickable
Upstairs room	tatami mats 1.9×0.95 with cloth border; plaster walls on timber posts; 4 fusuma doors; tea table with pot, 2 cups and flask; 3 cushions; futon with quilt; tansu chest with drawers; sideboard with radio; ceiling fan, stand fan; mirror	woven tatami texture; six pictures drawn from rectangles, ellipses, triangles and lines

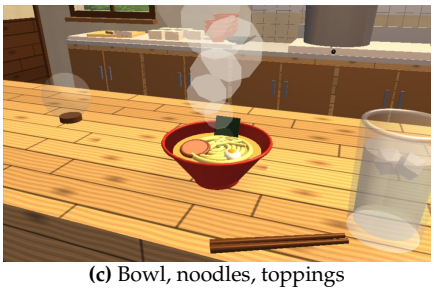
Table 6: The main objects of the shop and how each is built.



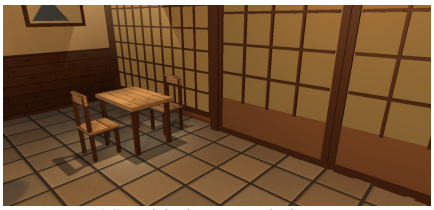
(a) Counter, stools, lamps



(b) Stove, pots, counter top



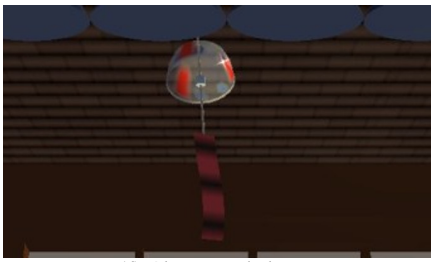
(c) Bowl, noodles, toppings



(d) Table lamp and chairs



(e) Porch: banner and chime



(f) Glass wind chime

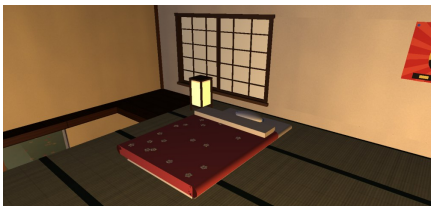
Figure 4: Some of the objects of the shop, each built from the primitives of Figure 3.



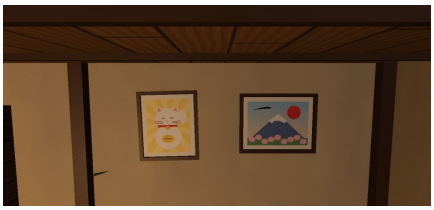
(a) Tatami room



(b) Windows, lamp and fan



(c) Futon and andon lamp



(d) Pictures drawn in code



(e) Mirror on the front wall



(f) Room from the orbit camera

Figure 5: The upstairs tatami room.

three child branches at the first level and two at deeper levels, each at a random angle, and the tips carry leaves or flowers. A tree is built once into OpenGL display lists (a trunk and seven branch groups); each frame the seven groups are rotated about their own base by a small wind angle, so the crown moves like a gust is passing and no two trees move together. Each cherry tree sheds 14 falling petals and each maple 12 leaves, and a carpet of

Tree (placed)	Trunk and branching	Crown	Variants
Cherry blossom (7)	2.05 m trunk, 5 roots, 3 levels (17 branches per group)	5–6 five-petal flowers per tip, a faint pink glow at night, buds and leaves	3 + 3 distant
Maple (6)	slender 2.5 m trunk, long spiky roots, 4 levels (53 per group)	5–7 five-pointed leaves per tip in red and orange	3 + 3 distant
Broadleaf (6)	flared 3.1 m trunk, steep branches, 3 levels	pads of 10 leaf cards per tip	3 + 1 light + 3 distant
Pine (5)	3.3 m trunk, 4 roots	6 tiers of jagged cone skirts, darker low and lighter high, with a spire	3
Bamboo (2 groves)	9 culms on a spiral, 4.6–6.6 m, 9–11 nodes	pale node bands with dark rings, leaf tufts at the top	3 + 3 distant
Jungle patch (8)	6 dome bushes (260 blades each) and 6 fern fronds	ground cover	1

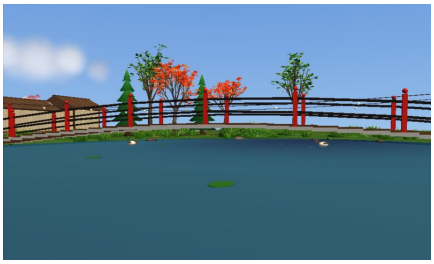
Table 7: The tree types, how many of each are placed, and how they differ.



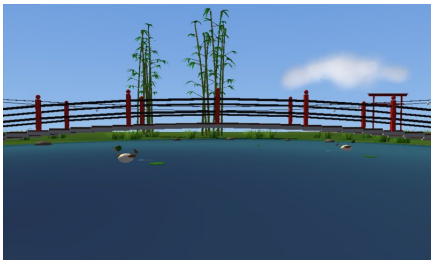
(a) Cherry blossom



(b) Maple



(c) Pine and maple



(d) Bamboo and torii gate



(e) Pines, bushes and houses



(f) Grass tufts

Figure 6: The kinds of tree and the undergrowth, in daylight.

petals or leaves lies under it. The lake is a semi-transparent disc with a muddy bank, 12 shore rocks and 4 lily pads, crossed by a 17 m arched bridge of 34 planks with red posts and rails and a string of small lights. Two ducks swim on a circle of radius 3.5 m, always facing their direction of travel, and fireflies drift over the grass at night. The ten houses come in two designs (Figure 7): a 4×3.5 house with a veranda and four red lanterns, and a wider 6×3.5 house on a raised floor with sliding shoji panels and a side porch.

3.5 3D transformations

All placement goes through one function that applies the standard transform to the modelview matrix:

$$M = T(\mathbf{p})\, R_x(\alpha)\, R_y(\beta)\, R_z(\gamma)\, S(s_x, s_y, s_z), \tag{1}$$

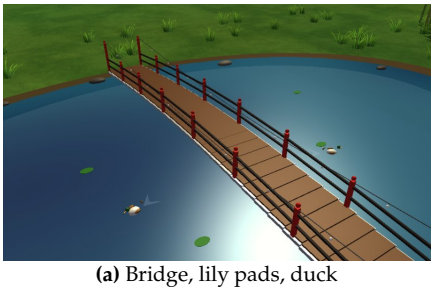
so a part is scaled first, then rotated, then moved. Nesting parts with the matrix stack gives hierarchical models: a child inherits everything its parent does. Table 8 lists where each transformation is used and Figure 8 shows the joint hierarchies. For the cyclist the knee is found from the hip $\mathbf{h}$, the pedal $\mathbf{f}$ and the bone lengths L_1, L_2 with the law of cosines. With $d = |\mathbf{f} - \mathbf{h}|$ (at most $L_1 + L_2$),

$$a = \frac{L_1^2 - L_2^2 + d^2}{2d}, \qquad h = \sqrt{L_1^2 - a^2}, \qquad \mathbf{k} = \mathbf{h} + a\,\hat{\mathbf{u}} + h\,\hat{\mathbf{u}}^{\perp}, \tag{2}$$

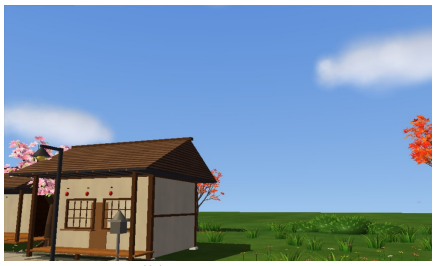
so the knee always bends forward and the shin follows the pedal exactly. A wheel of radius r that has rolled a distance d has turned by $\theta = d/r$, and the cyclist moves along the street by $x(t) = x_0 + (v\,t \bmod L)$.

3.6 Textures and texture mapping

There are 13 procedural textures, each painted by a function when the program starts (Table 9). Most are made from *value noise* summed over several octaves (fractal Brownian motion) and wrapped so that they tile. The noise



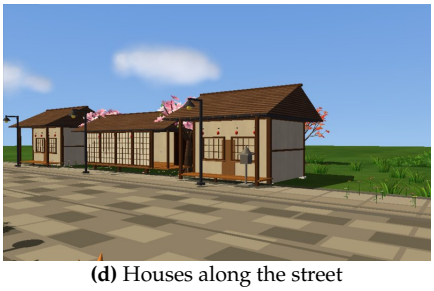
(a) Bridge, lily pads, duck



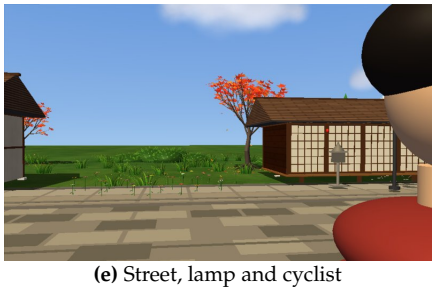
(b) House type 1



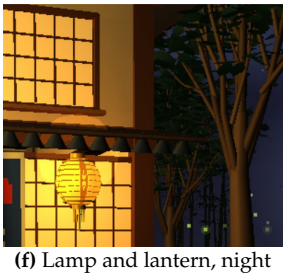
(c) House type 2



(d) Houses along the street



(e) Street, lamp and cyclist



(f) Lamp and lantern, night

Figure 7: Buildings, water and street lighting of the landscape.

Transformation	Used for
Translation	cyclist, walking person and cat, sliding shoji doors, rising steam, ducks on a circle
Rotation	wheels (rolling without slipping), pedals and cranks, fan blades, hinged door, wind-chime pendulums, head of the stand fan swinging $\pm 35^\circ$
Scaling	cubes stretched into planks and counters; cat $\times 1.8$; bicycle $\times 1.4$; user scaling of selected objects (keys , and .)
Rotation about a pivot	$T(\mathbf{c})R(\theta)T(-\mathbf{c})$ for swaying tree branches, noren, hanging lanterns, the banner's top bar
Projection (shear)	flattening the scene onto the ground for shadows (Section 3.11)

Table 8: Where each 3D transformation is used.

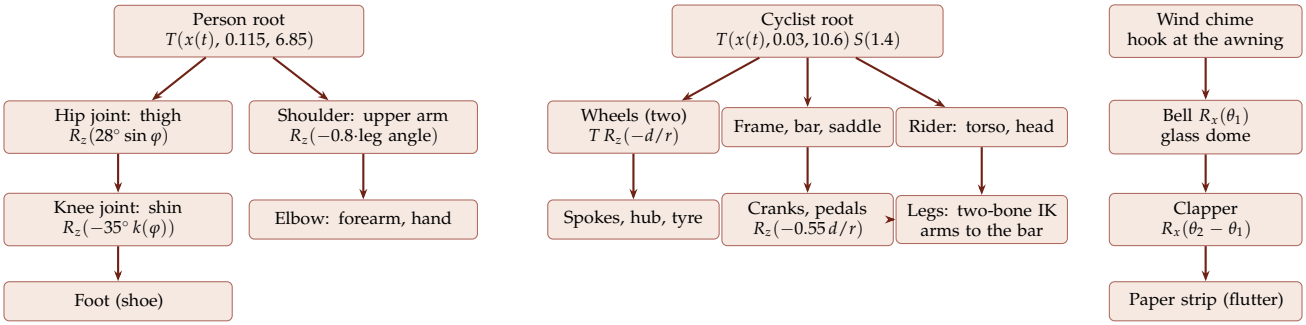


Figure 8: Parent–child transformations of the walking person, the cyclist and the wind chime. A joint moves everything below it; the dashed arrow means the rider's feet follow the pedals.

is a smoothly interpolated random lattice and the fractal sum adds finer octaves at half the strength:

$$n(x,y) = \mathrm{lerp}\left(\mathrm{lerp}(h_{00},h_{10},s_x),\mathrm{lerp}(h_{01},h_{11},s_x),s_y\right), \quad s(t) = 3t^2 - 2t^3, \qquad \mathrm{fBm}(x,y) = \sum_{k=0}^{m-1} 2^{-(k+1)}\, n(2^k x,\, 2^k y). \tag{3}$$

They are used with the lighting (the light darkens and tints them), with mipmaps and anisotropic filtering so that they do not shimmer, and with planar mapping: the texture coordinates of a face are its size divided by a scale, so a plank is always the same width whatever the box. **Grass.** The lawn is one big quad drawn without lighting. The

Texture	How it is painted	Used for
Wood planks, dark wood	noise grain with plank seams, two colours	counter, floors, table tops, doors, posts
Glazed floor tile	grid of tiles with grout and gloss	dining floor
Lime plaster	soft low-contrast noise	walls inside and outside
Roof tile	rows of dark corrugated tiles	roofs of the shop and houses
Tatami	rounded reeds, row tone, fibres, mottling	upstairs floor
Grass (mine)	four greens by noise, blade strokes, grain; a large macro layer	the lawn
Water	blue gradient with soft ripples, scrolls with time	the lake
Cloud sprite	round noise puff with a soft edge (alpha)	cloud billboards
Sphere-map environment	a blurred picture of the room	faint reflection on pots and glass
Asphalt, concrete	grainy noise	prepared for stone surfaces

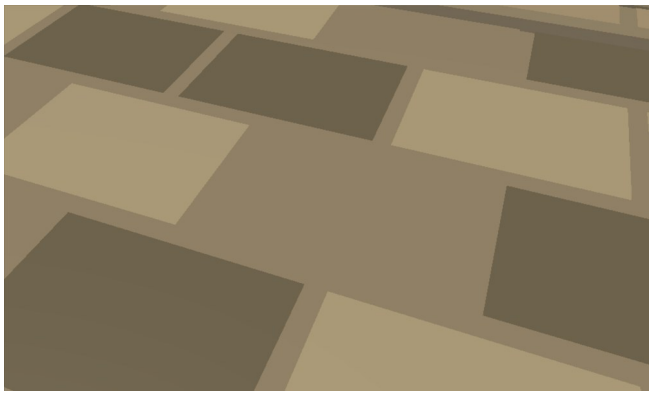
Table 9: The procedural textures. The cobblestone street is built from geometry with a hashed stone colour.

detail texture (512×512, repeated every 14 units) is made in three steps: four greens are mixed by noise; thousands of thin blade strokes (mostly upright, some sun-bleached or dead) are painted over them; a little grain is added. A second soft texture, repeated every 90 units, is multiplied in with the blend $C = 2\,C_{dst}C_{src}$ (neutral at 0.5), so the repeat cannot be seen. On top stand about 500 real grass tufts of 30 arching blades that sway in the wind.

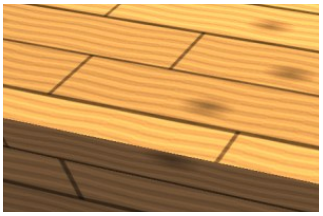
Cobblestone street. Stones of 1.2×0.8 are set in a brick pattern with thin grout; each stone is light, base or dark by a hash of its position. Raised curbs and a darker centre line finish the road, and the sidewalks use 1.4×1.4 flagstones in two tones.



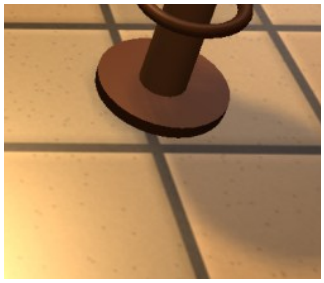
(a) Grass: texture, tufts and flowers



(b) Cobblestone street



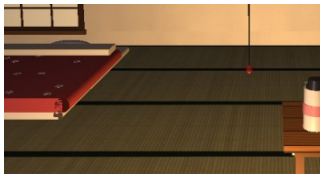
(c) Wood planks



(d) Glazed floor tile



(e) Tatami, cloth border



(f) Board ceiling, beams

Figure 9: The procedural textures as they appear in the scene.

3.7 Illumination: ambient, diffuse and specular light

OpenGL gives 8 lights, and each light i has its own ambient, diffuse and specular colour. The colour of a surface point with unit normal $\mathbf{N}$, light direction $\mathbf{L}$ and half vector $\mathbf{H} = \widehat{\mathbf{L}+\mathbf{V}}$ is

$$I = \mathbf{e} + \sum_i a_i \Big[\mathbf{c}_a \mathbf{l}_{a,i} + \mathbf{c}_d \mathbf{l}_{d,i} \max(\mathbf{N}\cdot\mathbf{L}_i, 0) + \mathbf{c}_s \mathbf{l}_{s,i} \max(\mathbf{N}\cdot\mathbf{H}_i, 0)^n \Big], \qquad a_i = \frac{\sigma_i}{k_c + k_l d + k_q d^2}, \tag{4}$$

where $\mathbf{e}$ is the emission of the material (lantern paper, bulbs), $\mathbf{c}_{a,d,s}$ are the material colours, n is the shininess and a_i the attenuation. For a spot light $\sigma_i = (\cos\alpha)^e$ inside the cone (cut-off 28°, $e = 35$) and 0 outside. The global ambient light is zero, so a lamp that is off really lights nothing.

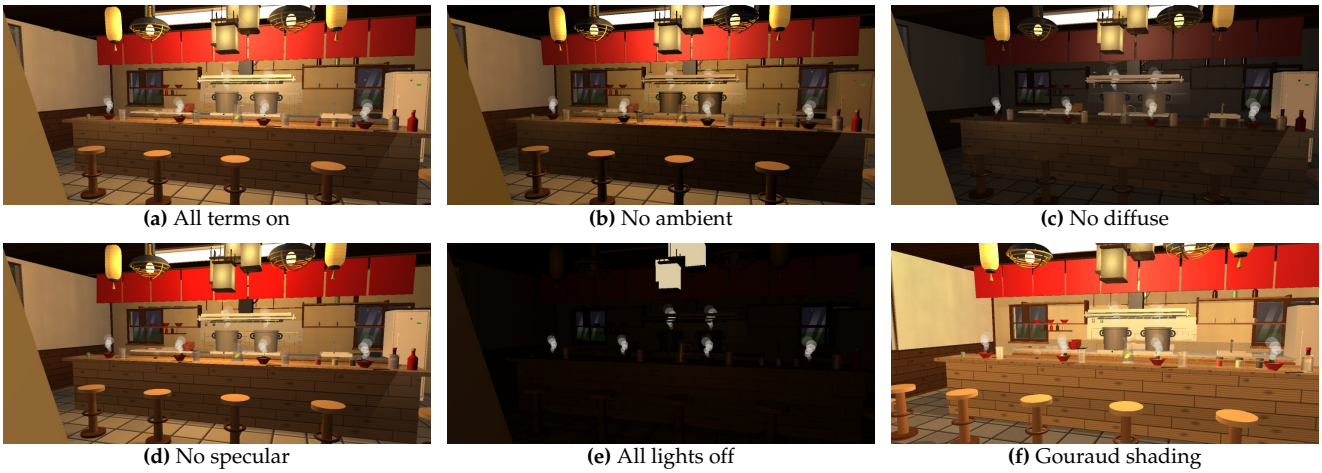
Only 8 lights exist, but the shop needs more, so the four lights that normally serve the outside (entrance lanterns, street lamps, sign) are re-assigned to interior roles while the inside is drawn (Table 10). Keys `1`–`7` switch each fixture, `0` the sun or moon, and `-`, `=`, `\` remove just the ambient, diffuse or specular term from every light.

Light	Type	Position	Role (outside / inside the shop)
0	directional	$(0.45, 0.70, -0.45)$	sun (warm white) or moon (blue-silver)
1	point, $k_{l,q} = 0.14, 0.07$	$(0, 2.85, 0)$	dining pendant lamps (key `1`)
2	point	$(-3.8, 2.5, 4.5)$ / $(0, 2.6, 4.7)$	left entrance lantern / light leaking through the paper wall
3	point	$(3.8, 2.5, 4.5)$ / merged	right lantern / box lanterns, hanging lanterns, ceiling panel (keys `3`–`5`)
4	spot 28°, $e = 35$	$(-2.4, 3.25, 2.8)$	left table lamp (key `2`)
5	point	$(0, 3.4, 7)$ / $(2.4, 3.25, 2.8)$	street lamps / right table lamp (key `2`)
6	point	$(0, 4.5, 4.2)$ / $(-4.1, 1.4, 3.2)$	shop-sign glow / floor lantern tower (key `6`)
7	point	$(0, 5.15, 0)$	upstairs paper lamp (key `7`)

Table 10: The eight lights. At night all fixtures glow in their own warm colour and flicker slowly.

Key	Fixture	Diffuse colour	Specular colour	Reach (k_l, k_q)
1	Dining pendant lamps	(0.70, 0.48, 0.18)	(1.00, 0.92, 0.76)	0.14, 0.07
2	Two table lamps (spot)	(0.90, 0.88, 0.78)	(1.00, 1.00, 0.95)	0.09, 0.032
3–5	Ceiling panel, box and hanging lanterns	$0.55 \times (1.00, 0.82, 0.56)$	$0.75 \times (1.00, 0.82, 0.56)$	0.15, 0.07
6	Floor lantern tower	$0.55 \times (1.00, 0.82, 0.56)$	$0.75 \times (1.00, 0.82, 0.56)$	0.28, 0.13
7	Upstairs paper lamp	(0.40, 0.30, 0.14)	(0.60, 0.52, 0.36)	0.22, 0.20
0	Sun (day) / moon (night)	(0.85,0.78,0.55) / (0.14,0.16,0.25)	(0.95,0.90,0.75) / (0.22,0.25,0.35)	directional

Table 11: The light values of each fixture.



(a) All terms on **(b)** No ambient **(c)** No diffuse

(d) No specular **(e)** All lights off **(f)** Gouraud shading

Figure 10: (a)–(e) The illumination terms switched off one at a time, at night: without ambient the shadows are slightly deeper, without diffuse only glow and highlights remain, without specular the shiny counter goes flat. (f) The same view with Gouraud shading.

3.8 Glossy surfaces and the specular term

The specular term makes a surface look glossy. For a light of specular colour $\mathbf{l}_s$ and a material of specular colour $\mathbf{c}_s$ and shininess n it adds

$$I_s = \mathbf{c}_s \, \mathbf{l}_s \, (\mathbf{N}\cdot\mathbf{H})^n \, a, \tag{5}$$

which is brightest when the half vector lines up with the normal, that is when the eye sees the mirror image of the lamp. The exponent n sets how narrow the highlight is (Figure 12): a small n gives a broad weak sheen (plaster,



(a) All lights on **(b)** All lights off **(c)** Only 1: pendants **(d)** Only 2: table lamps



(e) Only 3: ceiling panel **(f)** Only 4: box lanterns **(g)** Only 5: hanging lanterns **(h)** Only 6: lantern tower

Figure 11: Every light switched on on its own at night, in the same view (the moon is off). Only the surfaces near a lamp are lit by it.

cloth), a large n a small sharp spot (glass, glaze). Table 12 lists the materials. For ordinary materials the specular colour is white; for metals it is tinted by the surface colour, which is why brass looks gold and steel looks grey. Roughness r maps to shininess by $n = 4 + 124(1 - r)^2$.

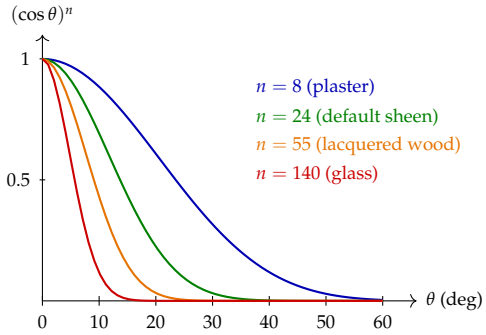


Figure 12: The specular lobe $(\cos\theta)^n$ for four materials, where θ is the angle between **N** and **H**: a larger n makes a narrower highlight.

Surface	Specular ρ_s	n	Look
Plaster walls, cloth, banner	0.03–0.05	8–10	matte, no visible highlight
Tatami	0.10	12	faint sheen along the rushes
Default surfaces	0.14	24	soft satin so nothing looks flat
Wainscot, shelves (varnish)	0.30–0.35	50–55	soft broad highlight
Counter body; chair legs	0.65–0.80	40–50	satin lacquer
Counter top, tables, stool seats	1.0	55–60	lacquer: a bright, clear highlight
Glazed floor tiles	0.75	55	shiny pools of light under the lamps
Bowl glaze; broth; egg	0.90; 0.90; 0.45	110; 85; 42	sharp, warm and soft highlights
Steel pots (dark body, white highlight)	1.0	90 (rim 120)	metal with a faint reflection of the room
Glass (tumbler, bell, lamp shades)	1.0	100–140	tiny, very bright highlights

Table 12: Specular strength and shininess of the materials of the shop.

3.9 Sky, day and night

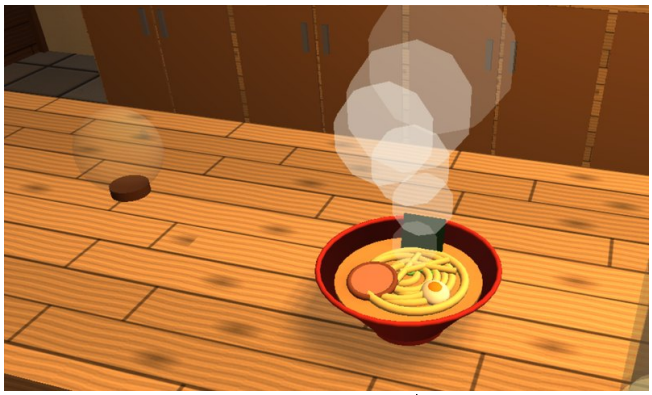
The sky is drawn first, with depth writes off and the camera translation removed, so it stays infinitely far away (the skybox technique) and the scene is drawn over it. Table 13 lists its layers. Glowing things add light, so halos use additive blending, $C = \alpha C_s + C_d$. The sky colour at a ring of elevation e, and the fog that blends distant things into it, are

$$C_{sky} = (1 - t)\, C_{horizon} + t\, C_{zenith}, \quad t = (e/90^\circ)^{0.55}, \qquad f = e^{-(\rho z)^2}, \quad C = f\, C_{surface} + (1 - f)\, C_{fog}. \tag{6}$$

Day and night (key `T`). One switch changes the sky, the sun or moon light (warm gold by day, cool blue at night), the glow of every lantern and window (dimmed to about 15–20% by day), the street lamps, the fog density (0.012 or 0.022), the fireflies and the sun shadows (day only).



(a) Specular on



(b) Specular off (key \)

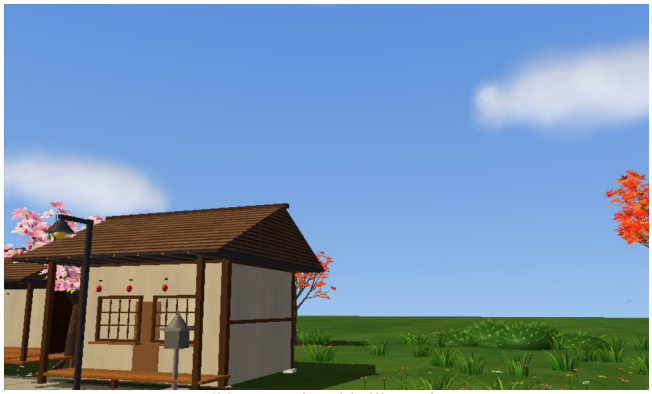
Figure 13: The lacquered counter top with and without the specular term, looking down at the lamps. The highlight is the mirror image of the lamp in the lacquer.

Layer	Construction
Dome	half sphere of 14 rings; colour blends from horizon to zenith with $t = (e/90°)^{0.55}$, e = elevation. Day: pale blue to deep blue; night: dark navy. By day the horizon equals the fog colour, so far ground melts into the sky
Sun	5 camera-facing discs with soft additive halos, warm yellow to white
Moon	the same place at night: cool halos, a disc and six darker craters
Stars	132 points (57 bright, 75 dim) that twinkle with their own speed
Clouds	12 clouds of 22 billboards each, one soft sprite, flat base and domed top, drifting with time

Table 13: The layers of the sky.



(a) Day: sun glow and blue sky



(b) Day: cloud billboards



(c) Night: cratered moon, fireflies, lamp



(d) Night: stars and dark clouds

Figure 14: The sky by day and at night.

3.10 Gouraud and Phong shading

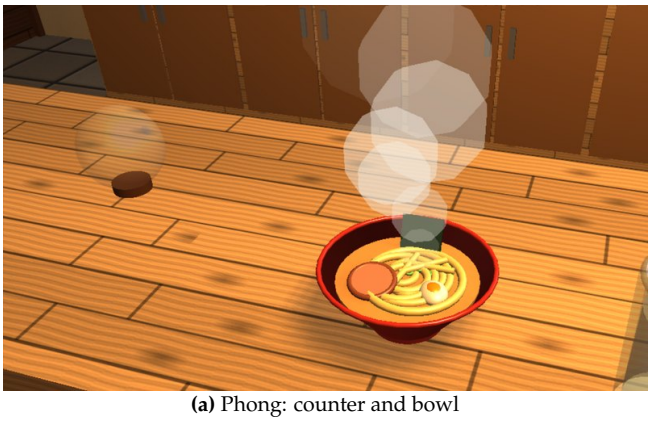
With key `G` the whole scene switches between two shading models.

- **Gouraud shading** (fixed function): the lighting equation is evaluated at each vertex and the colour is interpolated across the polygon, $C = \lambda_1 C_1 + \lambda_2 C_2 + \lambda_3 C_3$. It is fast, but a highlight that falls between vertices disappears.
- **Phong shading** (my GLSL 1.20 shader): the normal is interpolated, $\mathbf{N} = \lambda_1 \mathbf{N}_1 + \widehat{\lambda_2 \mathbf{N}_2} + \lambda_3 \mathbf{N}_3$, and the same equation is evaluated for every pixel, for all 8 lights, with spot cones, attenuation, fog and a per-light on/off mask.

Flat shading (one colour per polygon) was not implemented. Fog uses $f = e^{-(\rho z)^2}$. Table 14 compares the two models and Figure 15 shows the difference.

	Gouraud shading	**Phong shading**
Where the light is computed	at every vertex (fixed function)	at every pixel (GLSL fragment shader)
What is interpolated	the colour	the normal, re-normalised per pixel
Highlights, spot cones	lost when they fall between vertices	exact, smooth edges
Textures	multiplied with the interpolated light	multiplied before lighting is added
Cost	low	8 lights per pixel (a mask skips lamps that are off)

Table 14: Gouraud and Phong shading compared as implemented here.



(a) Phong: counter and bowl



(b) Gouraud: same view



(c) Phong: the dining area



(d) Gouraud: same view

Figure 15: The same views drawn with per-pixel Phong shading and with per-vertex Gouraud shading (key `G`).

3.11 Shadows

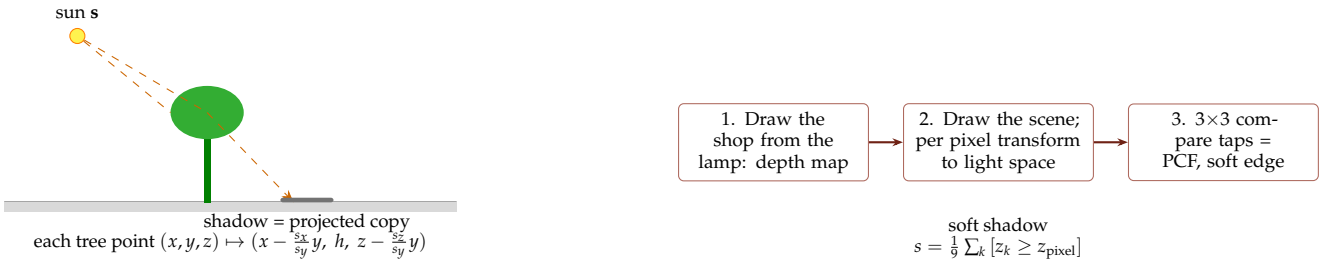
Three kinds of shadow are used (Figure 16).

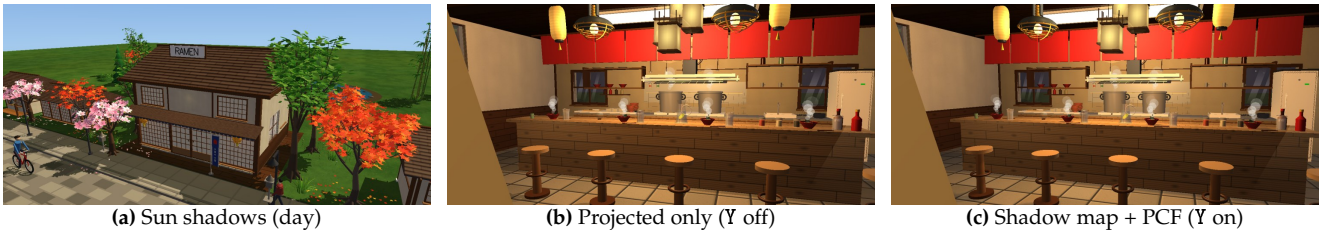
- **Outdoor sun shadows.** The whole outdoor scene is drawn a second time, flattened onto the ground along the sun direction **s** with a shear matrix that sends every point to

$$x' = x - \frac{s_x}{s_y} y, \qquad y' = h, \qquad z' = z - \frac{s_z}{s_y} y. \tag{7}$$

  The flattened drawing goes only into the stencil buffer, then one soft dark layer is blended over the marked pixels, so overlapping branches never darken twice. At night only a few simple fixed soft shadows are drawn.
- **Interior shadows.** The furniture is projected onto the tiled floor from the pendant and table lamps with the point-light version of the same matrix.

- **Soft shadows (key `Y`).** The counter, stools, tables, chairs and bowls are drawn from the pendant light into a 512×512 depth texture. In the shader every pixel is compared with the map 3×3 times and the results are averaged, $s = \frac{1}{9}\sum_k [z_L \le z_{map,k} + \epsilon]$ (percentage-closer filtering), so the edge is soft.



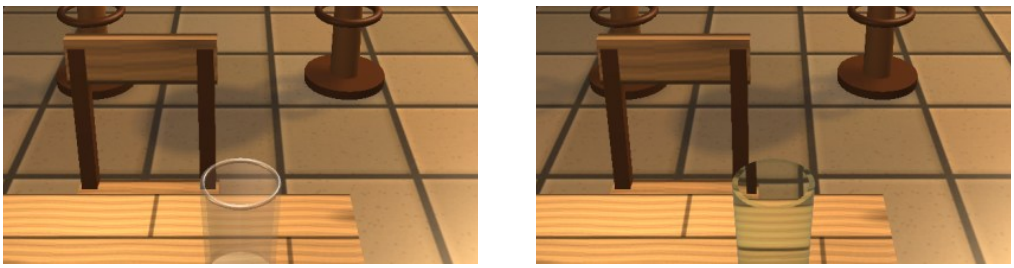


(a) Sun shadows (day) **(b)** Projected only (`Y` off) **(c)** Shadow map + PCF (`Y` on)

Figure 16: Top: how a tree is projected onto the ground, and the steps of soft shadow mapping. Bottom: projected tree, bicycle and person shadows; the dining area without and with the shadow map.

3.12 Glass refraction (screen space)

The empty tumbler on the left table is drawn last, over the finished picture. The screen area behind it is copied into a texture, and the tumbler is drawn with a shader that moves the lookup by the amount a ray bends at the glass (Snell's law, index 1/1.5), so the table and floor behind it appear shifted and distorted. A Fresnel reflection $F = 0.04 + 0.96(1 - \cos\theta)^5$ (Schlick) adds the shine of the room at grazing angles. It is an approximation: only what is already on the screen can be seen through the glass.



(a) `Z` off: plain glass **(b)** `Z` on: refraction

Figure 17: The tumbler on the left table without and with screen-space refraction.

3.13 Ray tracing

Key `R` adds a real ray tracer that runs on the CPU for eight featured objects (Table 15). The rest of the scene is still drawn by OpenGL; the ray tracer only decides the pixels of these objects (Figure 18).

Object	Count	Shapes	What the rays do
Steel stock pots	2	cone, top disk	reflect the kitchen, lamps and counter
Ramen bowls	4	foot ring, body, soup disk	glaze reflects the lamps; soup reflects weakly
Glass ball	1	sphere, radius 0.075	reflects and refracts (Snell, $n = 1.5$)
Mirror	1	slab 0.8×1.3 in a frame	near-perfect reflection of the upstairs room

Table 15: The ray-traced objects. All other shapes (floors, walls, counter, tables, lamps) exist only so that rays have something to hit.

- **Scene.** The shop is rebuilt for the rays from the same positions and sizes as the OpenGL models, with the same textures and the live OpenGL lights. Lamps are light-emitting shapes, so they show in reflections.
- **Intersection.** Exact formulas: slab test for boxes, quadratic equations for spheres and cones, plane test for disks. A **bounding volume hierarchy** (BVH) groups the small shapes in boxes, so a ray skips everything inside a box it misses.
- **Shading.** Surfaces are lit with the same Blinn–Phong model. Metal, glaze, soup and the mirror send a reflection

ray, up to three bounces; the glass ball bends rays with Snell's law:

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\mathbf{n}, \qquad n_1 \sin\theta_1 = n_2 \sin\theta_2, \qquad F = F_0 + (1 - F_0)(1 - \cos\theta)^5. \tag{8}$$

Reflection and refraction are weighted by the Fresnel term F; if the refracted ray does not exist the ray is totally reflected.

- **Speed.** Only the screen rectangles of these objects are traced, the rows are shared by up to 8 CPU threads, and a ray budget keeps the pass near 25 ms by tracing one ray per 2×2 block when a large area is in view. In a close view (about 7,000 eye rays) it takes about **13 ms in a Release build** and 80 ms in Debug.
- **Compositing.** The result is drawn over the OpenGL frame only where the depth buffer shows the same surface, so a nearer object still hides a pot or the ball.

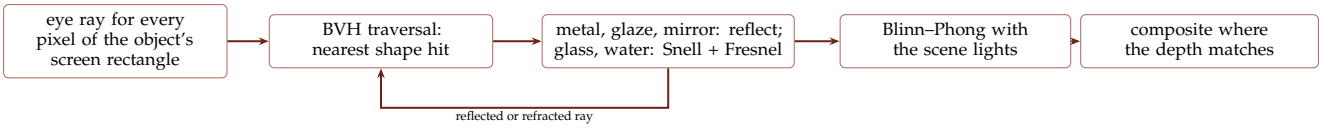


Figure 18: The steps taken for every eye ray of a ray-traced object.



(a) OpenGL only (R off)



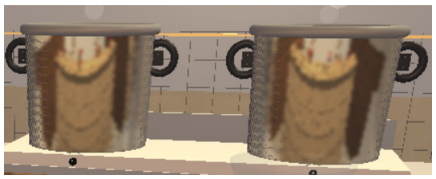
(b) Ray tracing on



(c) Bowl and glass ball



(d) Pots, OpenGL (zoom)



(e) Pots, ray traced (zoom)



(f) The mirror upstairs

Figure 19: Ray tracing (key R): the steel pots mirror the shop, the glass ball refracts the counter, and the mirror reflects the room. Up close one ray covers several pixels, so the result looks slightly blocky.

3.14 Animation

All motion runs from one clock, advanced by a 16 ms timer, so the space bar pauses everything at once (Table 16). **Wind chime physics.** The bell (angle θ_1, length 0.12 m) and the clapper with its paper strip (θ_2, length 0.20 m) are

Object	Motion
Cyclist	rides from $x = -16$ to 16 at 7 units/s and restarts; wheels turn by d/r (no slipping), cranks by $0.55\,d/r$, legs follow the pedals by two-bone IK; the headlight glows and throws a beam at night
Person	walks right to left at 1.8 units/s with 28° hip swing, knees that bend while the leg swings forward, arms swinging against the legs and a body bob
Cat	walks left to right at 3 units/s; every third lap it stops in front of the shop, turns, sits and washes its face (paw lick, wipe over the ear, head tilt) before walking on
Nobori banner	grid of quads displaced by a travelling wave, calm at the pole and stronger at the free edge
Wind chime	two coupled pendulums with real physics (below)
Fans	ceiling fan blades 160°/s; stand fan blades 900°/s, head swinging 35°
Nature	wind sway of every tree branch group, falling petals, ducks on circles, fireflies, ripples
Doors	entrance door hinge, sliding doors, opened by a mouse click

Table 16: The moving objects of the scene.

pendulums driven by a gusty wind $w(t)$ and integrated with 8 small steps per frame:

$$\ddot{\theta}_i = -\frac{g}{L_i}\sin\theta_i - c_i\dot{\theta}_i + k_i\, w(t), \qquad i = 1, 2. \tag{9}$$

The paper strip catches much more wind than the glass ($k_2 = 22$ against $k_1 = 3$), so the clapper swings further; when the relative angle reaches 0.42 rad it hits the glass, bounces back and a star-shaped glint flashes on the rim.



(a) Cyclist, with shadow



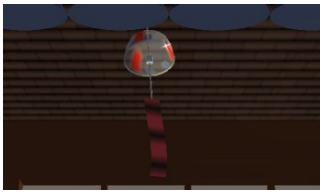
(b) Person and cyclist



(c) Person and cat



(d) The street with all three characters



(e) Wind chime



(f) Person on the sidewalk

Figure 20: The moving things in the street and the glass wind chime under the awning.

3.15 Cameras, interaction and controls

The world uses a perspective projection (45°, 62° in game mode); for field of view ϕ, aspect ratio a and clip planes n, f it is

$$P = \begin{pmatrix} \frac{1}{a\tan(\phi/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\phi/2)} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}, \tag{10}$$

so after the division by the depth, far objects look smaller. The game screens use an orthographic projection (no division by depth) so that the 2D overlay keeps its pixel size. A click is turned into a world ray and intersected with the plane of each door; the hit point is tested against the door's rectangle. Keys select an object (Tab) and move, rotate and scale it. Table 17 lists the cameras and Table 18 the controls.

Camera	Behaviour
Orbit (default)	orbits the shop; arrows, wheel and mouse drag change angle, distance and height
Walkthrough (`C`)	first person; `WASD` to move, mouse to look, stairs followed automatically
Counter, Upper Room, Pond (`C`)	orbit inside the counter area, the tatami room, and around the lake with the ducks
Bowl close-up (`B`)	orbits a ramen bowl at 0.7 m, a good place to see steam and glass
Light tour (`8`)	flies to the next lamp and switches it on; `9` toggles it

Table 17: The seven camera modes.

Key	Action	Key	Action
`C`, `B`, `8`, `9`	next camera, bowl close-up, light tour, toggle tour lamp	`T`	day / night
Mouse drag, wheel	orbit or look; zoom or height	`0`	sun / moon on / off
Arrows, PgUp / PgDn	orbit, zoom, higher / lower	`1`–`7`	each lamp on / off
`W A S D`, `Q E`	walk; fly down / up	`-`, `=`, `\`	ambient / diffuse / specular
Left click	open or close a door	`P`	lighting presets
`Tab`, `I J K L U N`	select; move an object	`G`, `Y`, `Z`, `R`	Phong / Gouraud, soft shadows, refraction, ray tracing
`[ ] ; ' , .`	rotate, tilt, scale it	`H`, `O`, `F`, `X`	roof, outlines, fog, steam
`Space`, `Esc`	pause, quit	`Enter`, `M`	start the game, back to exploring

Table 18: The keyboard and mouse controls.



(a) Orbit



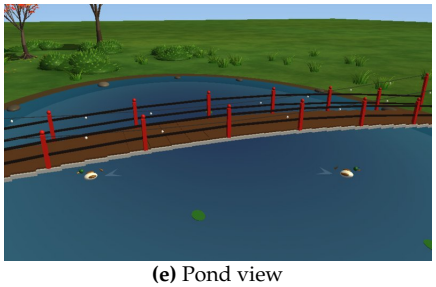
(b) Walkthrough



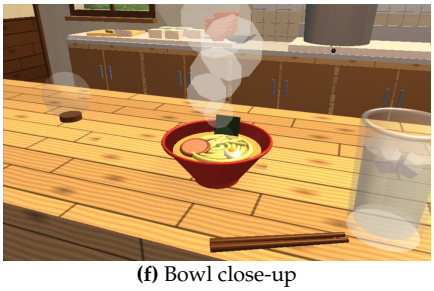
(c) Counter view



(d) Upper room



(e) Pond view



(f) Bowl close-up

Figure 21: Six of the seven camera modes.

3.16 The ramen cooking game

The game is a thin layer on top of the shop: a state machine, four stations, a few animated 3D objects and a 2D HUD. It is opt-in: the program starts in explore mode and `Enter` opens the title screen. A customer walks in and sits down and an order appears; the player takes raw noodles to the noodle pot, takes a bowl, scoops the noodles out when they look perfectly boiled, ladles broth, adds the toppings and serves the bowl (Figure 22). The noodle colour changes smoothly through five states: raw (under 3 s), undercooked, perfect (6–9 s), overcooked and ruined (over 12 s). The order lists the required toppings (level 1: green onion; from level 2 pork, egg and onion at random;

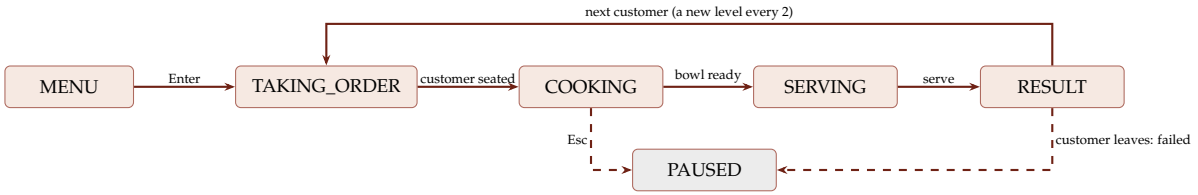
Station	Position, reach	What `E` does
Prep area	$x = -2.2$, reach 2.7	takes what is selected with keys 1–6: raw noodles, an empty bowl, or a topping
Noodle pot	$x = -0.55$, reach 2.5	with raw noodles: drops them in and starts the boil; with a bowl: scoops the noodles out
Broth pot	$x = 0.55$, reach 2.5	ladles tonkotsu broth into the bowl in hand
Serve counter	$z = -0.6$, reach 1.5	hands the bowl to the customer

Table 19: The four stations of the game.

from level 3 also seaweed). The bowl is graded by

$$Q = 0.5\,\text{accuracy} + 0.35\,\text{cook} + 0.15\,\text{time}, \qquad \text{stars} = \text{round}(5Q), \tag{11}$$

where accuracy is the matched share of required items minus extras, *cook* is $0.15, 0.60, 1.00, 0.60, 0.10$ for the five noodle states, and *time* is the patience left. Money earned grows with Q and the level; after every two customers the level rises (up to 4), which adds toppings and shortens the patience.





(a) Title screen



(b) Order board and prompts



(c) A customer at the counter

Figure 22: The game: its state machine and three screens (HUD drawn with an orthographic projection).

3.17 Keeping it fast

The scene has thousands of leaves, blades and planks, so several techniques keep the program interactive (Table 20).

Technique	What it does
Display lists	trees, bamboo, bushes, grass and petal carpets are compiled once; a frame only issues a few transforms
Frustum culling	an object is skipped when it is outside any of the six view planes
Distance LOD	beyond 18 m trees and bushes use a lighter build (one branch level fewer, fewer but larger flowers); bamboo beyond 24 m
Cheap shadows	the sun-shadow pass uses solid crown spheres for far trees and skips grass, petals and small details
Stencil shadows	one blend per pixel however many objects overlap
Textures	generated once, with mipmaps and anisotropic filtering, shared by every box of the same material
Shader	one program, a mask that skips lights that are off, glow switched off when no lamp is on
Ray tracing	only the screen rectangles of the featured objects, a BVH, all CPU threads and an adaptive ray budget

Table 20: The optimisations used to keep the scene interactive.

4 Results

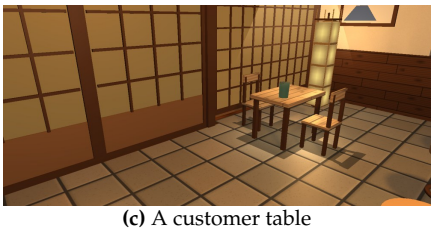
The program runs the whole shop, its surroundings and the game interactively, with every light, effect and animation described above switchable at run time. Figure 23 shows the finished scene by day and at night, inside and upstairs; Table 21 summarises the project and Table 22 lists parameters that can be changed to see their effect.



(a) The shop by day



(b) The shop at night



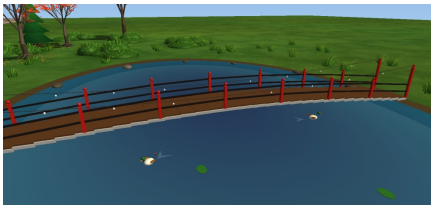
(c) A customer table



(d) Counter, stools, kitchen



(e) Tatami room



(f) Lake and ducks

Figure 23: The finished scene.

Item	Value	Item	Value
Lights / switchable fixtures	8 / 7	Camera modes	7
Procedural textures	13	Shadow map	512 × 512, PCF 3 × 3
Trees / groves placed	24 / 2	Moving characters	3 (cyclist, person, cat)
Houses, street lamps	10, 12	Ray-traced objects	8
Cooking-game stations	4	Ray-trace cost (Release / Debug)	about 13 ms / 80 ms
Source files, lines of code	35, about 15,000	External assets	none

Table 21: Scene statistics.

5 Discussion

What worked. Building every object from a few primitives kept the code small and the scene consistent, and the per-light switches made the lighting model easy to inspect. The stencil trick gives clean, non-overlapping tree shadows cheaply, and the mix of rasterisation with a small ray tracer shows real reflection and refraction without tracing the whole scene.

Problems and how they were solved.

Parameter	File	What changing it does
Shadow-map size (512)	`shadowmap.cpp`	1024 gives sharper soft shadows at a higher GPU cost
Spot cut-off 28°, exponent 35	`lighting.cpp`	wider cone / softer centre of the table lamps
k_l, k_q of each light	`lighting.cpp`	how far each lamp reaches (pool of light)
Ray-trace target time (25 ms)	`rtpass.cpp`	smaller: smoother frame rate but blockier ray tracing
Tree LOD distance (18)	`exterior.cpp`	distance at which trees switch to the lighter build
Wind-chime gains k_1, k_2	`scene.cpp`	how strongly the wind pushes the bell and the strip
Cyclist, person, cat speeds	`scene.cpp`	how fast they move along the street

Table 22: Parameters that can be changed to see their effect.

- *Only 8 lights.* The outdoor lights are re-assigned to interior fixtures while the inside is drawn, and a shader mask keeps outside lights from lighting the interior through the walls.
- *Lights switched off but the room still lit.* A global ambient and glowing materials caused it; the global ambient is now zero and glowing surfaces fade with their lamp.
- *Broken, streaky shadows.* The shadow planes lay a few millimetres above the ground and fought with it (z-fighting); depth testing is off for that pass.
- *Blocky ray-traced objects up close.* The ray budget keeps the cost bounded, so a large object gets one ray per several pixels; the drinking glasses and the jug, which looked worst, are drawn by OpenGL, and the table tumbler keeps its screen-space refraction.

Limitations. Flat shading is not implemented; refraction of the tumbler is a screen-space approximation; ray tracing covers only eight interior objects and uses no shadow rays; the outdoor night shadows are simple fixed shapes; there is no sound. **Possible improvements:** cube-map shadow maps for each lamp, a shadow map for the sun, ray tracing on the GPU with a GPU BVH, and customers that walk into the shop by themselves.

6 Conclusion

The Ramen Shop is a complete, explorable 3D scene written from scratch with OpenGL. It implements the proposed features (transformations, hierarchical models, a perspective camera, ambient, diffuse and specular light, Gouraud and Phong shading, a skybox and procedural textures; the orthographic view is used for the game screens and flat shading was not implemented) and adds three kinds of shadow, screen-space glass refraction, real ray tracing with a BVH, animated characters, a physically simulated wind chime and a playable cooking game. All of it runs interactively from a single Visual Studio project with no external assets.

References

[1] B. T. Phong, *Illumination for computer generated pictures*, CACM 18(6), 1975.

[2] J. F. Blinn, *Models of light reflection for computer synthesized pictures*, SIGGRAPH 1977.

[3] H. Gouraud, *Continuous shading of curved surfaces*, IEEE Trans. Computers C-20(6), 1971.

[4] T. Whitted, *An improved illumination model for shaded display*, CACM 23(6), 1980.

[5] L. Williams, *Casting curved shadows on curved surfaces*, SIGGRAPH 1978.

[6] W. Reeves, D. Salesin, R. Cook, *Rendering antialiased shadows with depth maps*, SIGGRAPH 1987.

[7] C. Schlick, *An inexpensive BRDF model for physically-based rendering*, Eurographics 1994.

[8] I. Sutherland, G. Hodgman, *Reentrant polygon clipping*, CACM 17(1), 1974.

[9] D. Shreiner et al., *OpenGL Programming Guide*, Addison-Wesley; Khronos, *GLSL 1.20 specification*.

[10] D. Ebert et al., *Texturing and Modeling: A Procedural Approach*, Morgan Kaufmann.